

Sistemas Distribuídos – Engenharia de Telecomunicações

Chamada de Procedimento Remoto

Prof. Emerson Ribeiro de Mello

mello@ifsc.edu.br

Licenciamento



Slides licenciados sob [Creative Commons "Atribuição 4.0 Internacional"](#)

Motivação para RPC

Dificuldades na comunicação entre processos via sockets

Cliente em Java

- 1 Conecta no servidor
- 2 Lê do teclado dois long
- 3 Converte long para String
- 4 Envia as Strings
- 5 Recebe o resultado

Servidor em C

- 1 Recebe via sockets vetor de char
- 2 Converte char para long
- 3 Faz a soma dos dois long
- 4 Devolve resultado como vetor de char

Serialização

Define como os *bytes* serão organizados para que possam ser armazenados ou transmitidos pela rede

Motivação para RPC

Dificuldades na comunicação entre processos via sockets

- **Comunicação** entre processos clientes e servidores por meio de **troca explícita de mensagens**
 - Ex: Aplicação Java enviando objetos do tipo Mensagem

Mensagem: msg
+ cod : int = 100
+ corpo : String = "Jogador 1"

```
// send  
saida.writeObject(msg);  
  
//receive  
Mensagem resp = entrada.readObject();
```

Não provê a **transparência de acesso**

- *Esconder as diferenças para representação dos dados*
- *Como os dados serão representados em diferentes arquiteturas*

Inspiração para RPC

Chamada de procedimentos em C

```
#include <stdio.h>
void funcao_a(int *r) {
    (*r)++;
    .....
}

int main(void) {
    int v = 1;
    .....
    funcao_a(&v);
    .....
    return 0;
}
```

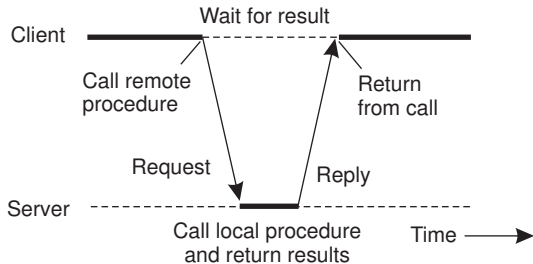
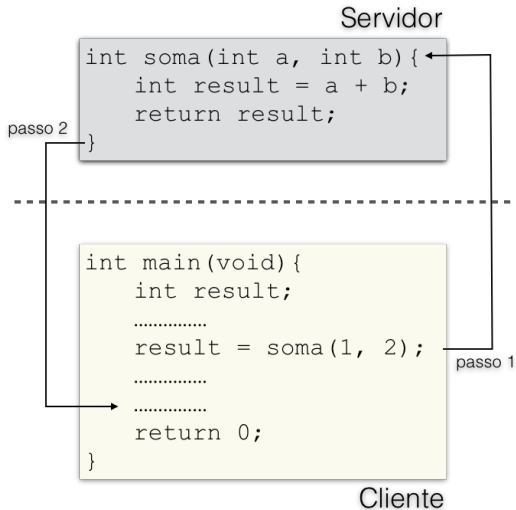
- **Transferência de controle e dados**

- Lista de parâmetros e tipos de retorno permitem a comunicação entre funções

- **Linguagem C: passagem de parâmetros pode ser por**

- Valor
 - Função que fora invocada cria uma cópia local da variável
- Referência
 - Função que fora invocada recebe endereço de memória da variável

Chamada de procedimento remoto (RPC)



- Chamada síncrona

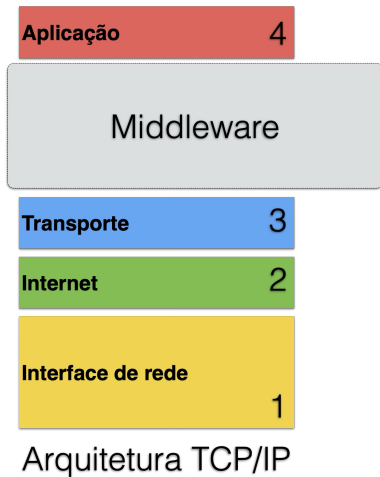
Chamada de procedimento remoto (RPC)



modelo OSI

- **Camada 6** – Representação e serialização dos dados
- **Camada 5** – Gerenciamento de conexão

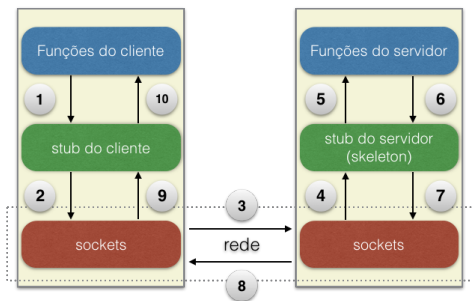
Chamada de procedimento remoto (RPC)



- **Camada 6** – Representação e serialização dos dados
- **Camada 5** – Gerenciamento de conexão

Chamada de procedimento remoto (RPC)

- Processos locais realizam chamadas de **procedimentos** que estão **localizados em outras máquinas**



- 1 Cliente invoca funções do stub
- 2 Stub serializa dados
- 3 Dados são transmitidos via sockets
- 4 Dados recebidos são entregues para o stub
- 5 Dados são deserializados
- 6 Resposta segue caminho inverso

Stub do cliente e Skeleton do servidor

- **stub do cliente**

- Atua como um *proxy*: para o cliente, a chamada parece local
- Empacota os parâmetros (serialização) e envia a requisição pela rede
- Recebe a resposta, desempacota os dados (deserialização) e devolve o resultado

- **skeleton do servidor**

- Recebe requisições de rede e despacha para a implementação do serviço
- Deserializa os parâmetros para encaminhar ao procedimento remoto
- Serializa o retorno da execução e envia a resposta ao cliente

Nota

Serialização define como os *bytes* serão organizados para que possam ser armazenados ou transmitidos pela rede

Representação dos dados

Marshalling

Conjunto de operações para transformar dados em memória em uma representação canônica para transmissão e reconstruí-los no destino

- **Marshalling** inclui serialização + metadados de tipo e estrutura
- Formato com **tipo implícito**
 - Somente os valores são transmitidos
 - Exemplo: XDR (*eXternal Data Representation*)
- Formato com **tipo explícito**
 - Além do valor, o tipo da variável também é transmitido¹
 - Exemplos: ASN.1, XML, JSON e Google Protocol Buffers

¹ASN.1 e Protocol Buffers enviam apenas o identificador do campo

Linguagem de Descrição de Interfaces

Interface Description Language (IDL)

- **Contrato do serviço remoto** onde se descreve operações, tipos de mensagens e formato dos dados
- Mantém independência de linguagem de programação e arquitetura de máquina
- Usada para gerar automaticamente *stubs* de cliente e servidor

Código 1: Exemplo de serviço definido em Protocol Buffers

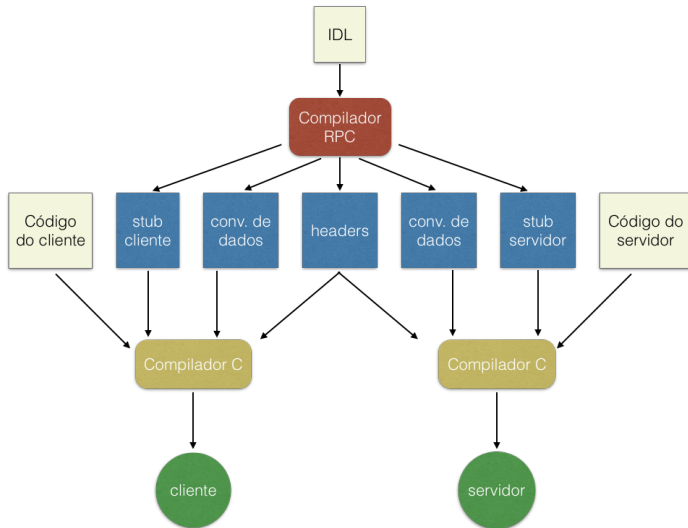
```
syntax = "proto3";  
service Agenda {  
    rpc Adicionar (Pessoa) returns (Resposta);  
    rpc Buscar    (Consulta) returns (Pessoa);  
}  
message Pessoa { int32 id = 1; string nome = 2; }  
message Consulta { int32 id = 1; }  
message Resposta { bool ok = 1; string msg = 2; }
```

Implementações de RPC

- **1984 – Sun RPC (ONC RPC)**
 - Primeiro sistema RPC comercial de sucesso, base do protocolo *Network File System* (NFS)
- **1996 – DCOM & Object RPC**
 - Lançado com o Microsoft Windows NT 4.0
- **2000 – SOAP RPC**
 - Uso do protocolo HTTP e o formato XML para padronizar chamadas de métodos pela Internet
- **2016 – gRPC**
 - Uso do protocolo HTTP/2 para transporte eficiente e *Protocol Buffers* para serializar os dados de forma binária

Exemplo com ONC RPC

Passos para escrever cliente e servidor com ONC RPC



Como escrever programas com ONC RPC

- Descrever a interface dos procedimentos (IDL RPC)

```
/* Dois procedimentos que recebem uma struct, faz a computação e retorna o resultado (int) */
struct operandos{
    int a; int b;
};

program CALCULADORA_PROG{
    version CALCULADORA_VERSION{
        int SOMA(operandos) = 1;
        int SUB(operandos) = 2;
    } = 1; /* versão do procedimento */
}= 0x20000001; /* versão do programa */
```

- Compilar IDL para gerar stubs para cliente e servidor

```
rpcgen calculadora.x # ou rpcgen -a -C calculadora.x
```

Servidor

```
#include "calculadora.h"

int * soma_1_svc(operandos *argp, struct svc_req *rqstp){
    static int result;
    result = argp->a + argp->b;
    return &result;
}
```

Cliente

```
#include "calculadora.h"

int main(void){
    char *host = "127.0.0.1";
    CLIENT *clnt;
    operandos argumentos;
    int *resultado;

    clnt = clnt_create(host, CALCULADORA_PROG, CALCULADORA_VERSION, "udp");
    if (clnt == NULL) { clnt_pcreateerror(host); exit(1); }

    argumentos.a = 1; argumentos.b = 2;
    resultado = soma_1(&argumentos, clnt); /* invocando procedimento remoto */
    printf("Resultado da soma: %d", *resultado);
}
```

Compilando e executando

Servidor

```
gcc servidor.c calculadora_svc.c calculadora_xdr.c -lnsl -o servidor
```

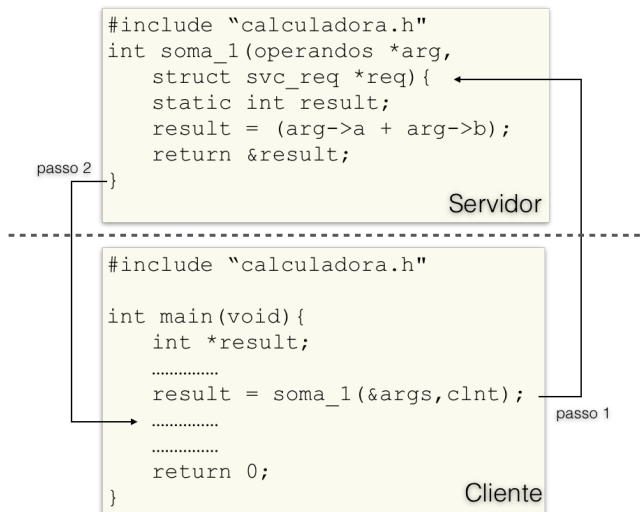
```
./servidor
```

Cliente

```
gcc cliente.c calculadora_clnt.c calculadora_xdr.c -lnsl -o cliente
```

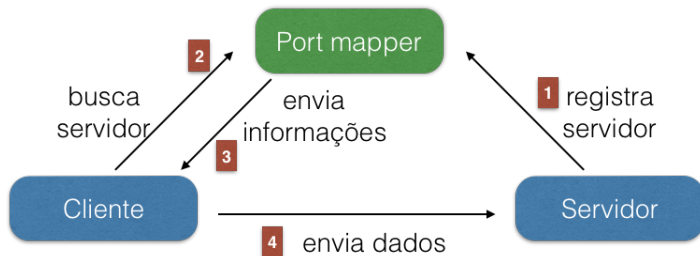
```
./cliente
```

Exemplo com ONC RPC



Exemplo com ONC RPC

Como cliente descobre a porta que o servidor está ouvindo



- **Port mapper** é um serviço de descoberta que permite aos clientes descobrirem em qual porta TCP/UDP o servidor está ouvindo
 - Executa por padrão na porta 111 (TCP ou UDP)
 - Versões 3 e 4 são chamadas de `rpcbind` protocol

Outras implementações de RPC

- Contrato de serviço definido em `.proto` (Protocol Buffers) que permite a geração automática de código para cliente e servidor
- Transporte sobre HTTP/2 permite a multiplexação de conexões e comunicação *full-duplex*

```
syntax = "proto3";

service Agenda {
  rpc Adicionar (Pessoa) returns (Resposta);
  rpc Buscar    (Consulta) returns (Pessoa);
}

message Pessoa { int32 id = 1; string nome = 2; }
message Consulta { int32 id = 1; }
message Resposta { bool ok = 1; string msg = 2; }
```

gRPC: tipos de chamada remota

- **Unary RPC** (1 requisição → 1 resposta)
 - Consultas simples, autenticação, operações CRUD
- **Server streaming RPC** (1 requisição → múltiplas respostas)
 - Monitoramento em tempo real e envio de eventos
- **Client streaming RPC** (múltiplas requisições → 1 resposta)
 - *Upload* em blocos, telemetria e agregação de dados
- **Bidirectional streaming RPC** (requisições e respostas em paralelo, nos dois sentidos)
 - Chat, colaboração e controle de dispositivos em tempo real

SOAP RPC

- **SOAP** baseada em troca de mensagens XML com foco na integração de aplicações
- Modelo clássico de *request-reply*, comum em *Web Services*
- Contrato descrito em **WSDL** (operações, tipos e *endpoints*)

Código 2: Trecho de um documento WSDL

```
<wsdl:definitions targetNamespace="http://localhost/Agenda.jws">

  <wsdl:message name="buscarPessoaRequest">
    <wsdl:part name="id" type="xsd:int"/>
  </wsdl:message>

  <wsdl:message name="buscarPessoaResponse">
    <wsdl:part name="id" type="xsd:int"/>
    <wsdl:part name="nome" type="xsd:string"/>
  </wsdl:message>
```

Conteúdo de mensagens transportadas com SOAP e gRPC

Formato textual explícito vs formato binário parcialmente implícito

- O conteúdo da mensagem SOAP do exemplo tem o tamanho aproximado de *294 bytes*

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:ag="http://localhost/Agenda.jws">
  <soapenv:Header/>
  <soapenv:Body>
    <ag:buscarPessoaRequest>
      <ag:id>12345</ag:id>
    </ag:buscarPessoaRequest>
  </soapenv:Body>
</soapenv:Envelope>
```

- Com gRPC, para `id = 12345`, apenas o valor é enviado, juntamente com o identificador (*tag*) do campo, em formato binário, tendo um tamanho aproximado de *8 bytes*
 - Parcialmente implícito pois só o número do campo (*tag*) é enviado no conteúdo da mensagem

Curiosidade

- Em <http://www.http2demo.io> tem uma demonstração comparando HTTP/1.1 com HTTP/2

Remote Method Invocation (Java RMI)

Objetos distribuídos e invocação de métodos remotos

- **Chamada Remota de Procedimento – RPC**
 - Permite que processos locais realizem chamadas de procedimentos que estão localizados em outras máquinas

Objetos distribuídos e invocação de métodos remotos

- **Chamada Remota de Procedimento – RPC**

- Permite que processos locais realizem chamadas de procedimentos que estão localizados em outras máquinas

- **Invocação de Métodos Remotos – Java RMI**

- **Semelhante ao RPC**, porém voltado para **objetos distribuídos**
- Estende o conceito de referência de objetos para um ambiente global distribuído
 - Objeto residente em um processo pode invocar métodos de um objeto residente em um outro processo
- Oferece *distributed garbage-collection*

RPC vs RMI

- Semelhanças entre RPC e RMI
- Diferenças entre RCP e RMI

RPC vs RMI

- **Semelhanças entre RPC e RMI**
 - Fazem uso de linguagem de descrição de interfaces (IDL)
 - São construídos sobre protocolos pedido e resposta
 - Oferecem o mesmo nível de transparência ao desenvolvedor
- **Diferenças entre RCP e RMI**

RPC vs RMI

- **Semelhanças entre RPC e RMI**
 - Fazem uso de linguagem de descrição de interfaces (IDL)
 - São construídos sobre protocolos pedido e resposta
 - Oferecem o mesmo nível de transparência ao desenvolvedor
- **Diferenças entre RCP e RMI**
 - Com o RMI o desenvolvedor poderá usufruir das facilidades da POO
 - objetos, classes, herança, ...
 - Objetos possuem uma referência única em todo o sistema
 - Objetos podem ser passados por referência (e não somente por valor)

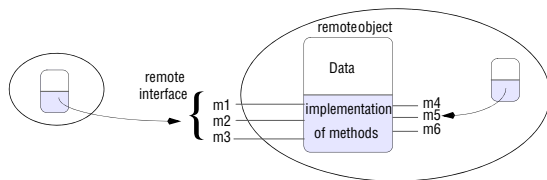
Modelo de objetos distribuídos

- **Referência de objeto remoto**

- um objeto só poderá invocar métodos de outros objetos que este possuir a referência

- **Interface remota**

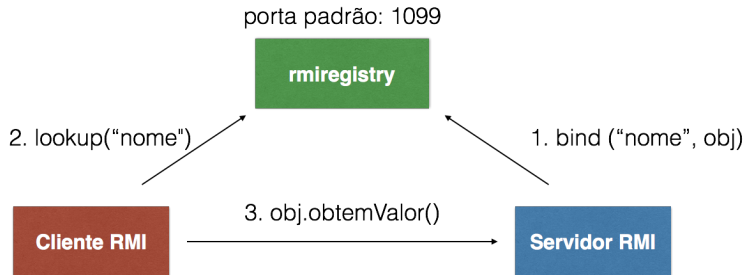
- cada objeto remoto possui uma interface que especifica quais de seus métodos poderão ser invocados remotamente
- objetos dentro de um mesmo processo poderão invocar os métodos remotos e também os locais



Entidades participantes

- **Servidor**
 - Processo que oferta objeto remoto
- **Cliente**
 - Processo que invoca um método de um objeto remoto
- **Serviço de registro**
 - Serviço de nomes que associa nomes a objetos
 - Ex: `rmi://host:port/pathName`

Java RMI



- Serviço de registro pode ser iniciado na linha de comando:

```
rmiregistry 12345 &
```

- ou dentro do código Java do processo servidor:

```
int PORTA = 12345;  
Registry registro = LocateRegistry.createRegistry(PORTA);
```

Remote Method Invocation (Java RMI)

Exemplo: Um Contador

Descrição da interface

```
package std;  
  
public interface ContadorDistribuido extends Remote{  
  
    public void incrementa() throws RemoteException;  
  
    public int obterValor() throws RemoteException;  
  
}
```

Atenção

Essa interface Java deve ser compartilhada entre servidor e clientes. Deve-se manter o mesmo nome de pacote Java.

Implementação da Interface no Servidor

```
package std;

public class Contador implements ContadorDistribuido{
    public int valor = 0;

    public void incrementa() throws RemoteException {
        this.valor++;
    }

    public int obterValor() throws RemoteException {
        return this.valor;
    }
}
```

Implementação do Servidor

```
package std;

public class Servidor{

    public static void main(String[] args) throws Exception{
        Contador c = new Contador(); // criando objeto

        // 0, indica para JVM escolher uma porta aleatória
        ContadorDistribuido stub = (ContadorDistribuido) UnicastRemoteObject.exportObject(c, 0);

        // Criando registry. Substitua o IP pelo endereço do seu computador
        System.setProperty("java.rmi.server.hostname", "192.168.0.1");
        Registry registro = LocateRegistry.createRegistry(1099);

        // Registrando objeto com o nome MeuContador
        registro.rebind("MeuContador", stub);

        System.out.println("Servidor pronto!");
    }
}
```

Implementação do Cliente

```
package std;

public class Cliente{

    public static void main(String[] args) throws Exception{

        // Obtendo referência para o registro (tem que conhecer IP e porta)
        Registry registro = LocateRegistry.getRegistry("192.168.0.1", 1099);

        // Obtendo referência para o objeto instanciado pelo Servidor
        ContadorDistribuido stub = (ContadorDistribuido) registro.lookup("MeuContador");

        // invocando métodos do objeto remoto
        System.out.println("valor: " + stub.obtemValor());
        stub.incrementa();
        System.out.println("valor: " + stub.obtemValor());
    }
}
```

Aulas baseadas em



COULORIS, George; DOLLIMORE, Jean; KINDBERG, Tim. **Sistemas Distribuídos: Conceitos e projeto**. 5. ed.: Bookman, 2013. <https://app.minhabiblioteca.com.br/books/9788582600542/>.



KRZYZANOWSKI, Paul. **Distributed Systems – Rutgers University**. 2020.



TANENBAUM, Andrew S; STEEN, Maarten van. **Sistemas Distribuidos: Princípios e paradigmas**. 2. ed.: Prentice Hall, 2008.